

SMART CAMPUS EVENT MANAGEMENT SYSTEM

School of Computing

**Bachelor of Technology
in
Computer Science & Engineering
By**

AITHA VENKATA NAVEEN (VTU24498)

*10211CS224
Full Stack Application Development
Slot : E1*



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA**

April, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled "Smart Campus Event Management System" by Aitha Venkata Naveen (VTU24498) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr. Kaviarasan S.

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

April, 2026

Signature of Course Co-ordinator

Dr.Sumithra M.

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

April, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(AITHA VENKATA NAVEEN)

Date: / /

ABSTRACT

The Smart Campus Event Management System is a web-based application designed to simplify the management of college events such as workshops, seminars, and cultural programs. Traditional event management methods often involve manual registrations, poor communication, and inefficient attendance tracking.

This system provides a centralized platform where students can view events, register online, book seats, access venue locations through Google Maps, receive notifications and submit feedback. Administrators can create events, manage registrations, track attendance, and generate certificates.

The application is developed using Spring Boot, Thymeleaf, and MySQL. It improves efficiency, reduces manual work, and enhances the overall event management process in educational institutions.

The system also includes OTP-based verification to ensure secure event registration and ticket booking. During the registration or booking process, a one-time password (OTP) is sent to the user's registered email for authentication. The user must enter the correct OTP to complete the process, which helps verify the user's identity and prevents unauthorized access. This additional security layer enhances data pro-

tection, ensures valid registrations, and improves the overall reliability and trustworthiness of the system.

Keywords:

Event Management,

Spring Boot,

Thymeleaf,

MySQL,

QR Code,

Seat Booking,

Full Stack Development,

One-Time Password (OTP).

LIST OF FIGURES

1.1	System Architecture	5
3.1	Execution Procedure	16
4.1	Data Flow Diagram	21
5.1	Login Page	27
5.2	Dashboard	28
5.3	Event Listing	29
5.4	Seat Selection and Booking with OTP Verification . . .	30
5.5	Venue Location	31
5.6	Admin Panel	32

LIST OF TABLES

2.1	Feature Comparison of Existing Applications	8
2.2	System Comparison Based on Functionality	8
5.1	Comparison of Existing Applications	26
7.1	Mapping of Project to Complex Engineering Problem (CEP)	36
7.2	Mapping of Project to Complex Engineering Activities (CEA)	37
7.3	SDG Mapping for the Project	37

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
CI	Continuous Integration
CRUD	Create Read Update Delete
DBMS	Database Management System
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
MVC	Model View Controller
ORM	Object Relational Mapping
OTP	One-Time Password
QR	Quick Response
REST	Representational State Transfer
UI	User Interface

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	v
LIST OF TABLES	vi
LIST OF ACRONYMS AND ABBREVIATIONS	vii
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	2
1.3 Scope of the Project	3
1.4 Framework	4
2 SURVEY OF SIMILAR APPLICATIONS IN THE MAR- KET	6
2.1 Limitations of Existing Systems:	7

2.2	Our Proposed System Advantages:	7
3	FRAMEWORK DESCRIPTION	9
3.1	Frontend Implementation	9
3.1.1	Environment Setup	9
3.1.2	Structure of the Project	10
3.1.3	Execution Procedure	10
3.2	Backend Implementation	12
3.2.1	Environment Setup	12
3.2.2	Structure of the Project	13
3.2.3	Execution Procedure	14
3.3	Database Implementation	17
3.3.1	Database Configuration	17
3.3.2	Schema Structure	18
3.3.3	Tables Created	19
4	METHODOLOGIES	20
4.1	Project Architecture	20
4.2	DataFlow Diagram	20
4.3	Module 1: Authentication Module	22
4.4	Module 2: Event Management Module	22
4.5	Module 3: Smart Features Module	23
4.5.1	Advantages of this Project	24

4.5.2	Disadvantages	25
5	RESULTS AND DISCUSSIONS	26
6	CONCLUSION AND FUTURE ENHANCEMENTS	33
6.1	Conclusion	33
6.2	Future Enhancements	34
7	ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG	36
7.1	Mapping of the Project to Complex Engineering Prob- lem (CEP)	36
7.2	Mapping of the Project to Complex Engineering Ac- tivities (CEA)	36
7.3	SDG Mapping	37
8	REFERENCES	38

Chapter 1

INTRODUCTION

1.1 Introduction

Colleges and universities frequently organize events such as workshops, seminars, hackathons, cultural programs, symposiums, and guest lectures to improve student learning and engagement. Managing these events manually can create several challenges for both students and administrators. Traditional event management methods often involve paper registrations, manual attendance tracking, spreadsheet maintenance, and communication through notices. These methods are time-consuming and may result in duplicate registrations, seat overbooking, delayed notifications, poor coordination, and difficulty in generating certificates.

To overcome these challenges, educational institutions require an automated system that simplifies the complete event management process. The Smart Campus Event Management System is a web-based application developed to automate event registration, seat booking, attendance tracking, and communication. Students can view available events, register online, book seats in real-time, access venue locations

through Google Maps, receive notifications, and submit feedback. The system also enhances security by implementing OTP-based verification during registration and booking processes. Administrators can create events, manage registrations, verify attendance using QR codes, and generate certificates for participants. The application is developed using Spring Boot, Thymeleaf, MySQL, and Spring Security to provide a secure and efficient solution for modern campus event management.

1.2 Aim of the Project

The aim of this project is to develop a smart web-based event management system that automates the complete process of organizing campus events in educational institutions. Colleges regularly conduct workshops, seminars, hackathons, cultural programs, and technical events, but managing them manually often leads to issues such as duplicate registrations, poor communication, inefficient attendance tracking, and seat allocation problems. This project aims to solve these challenges by providing a centralized digital platform for managing events efficiently.

The system is designed to simplify event registration, real-time seat booking, attendance tracking, certificate generation, venue nav-

igation through Google Maps, and communication between students and administrators. It also allows students to receive notifications, access event details, and submit feedback after participation. The project helps reduce manual effort, save time, improve operational efficiency, and enhance the overall user experience in campus event management.

1.3 Scope of the Project

The scope of this project is to provide an efficient and automated platform for managing various events conducted in colleges, universities, training institutions, and other organizations. The system can be used for managing academic events such as workshops, seminars, symposiums, hackathons, guest lectures, and technical competitions, as well as non-academic events like cultural programs and sports activities. It simplifies tasks such as event creation, student registration, seat booking, attendance tracking, certificate generation, and feedback collection.

The project can be further extended to support online webinars, virtual events, and external event participation. Additional features such as online payment integration, mobile application support, advanced analytics, and AI-based event recommendations can also be implemented in the future. This makes the system highly flexible and

suitable for modern event management requirements.

1.4 Framework

The project follows the Model View Controller (MVC) architecture to ensure proper separation of components and efficient system management. The frontend is developed using Thymeleaf templates along with HTML, CSS, JavaScript, and Bootstrap to create an interactive web interface for students and administrators. The backend is developed using Spring Boot to handle business logic such as event registration, seat booking, attendance tracking, QR code generation, certificate generation, and notifications. Spring Security is implemented to provide secure authentication and role-based access control. OTP-based verification is implemented as part of the authentication mechanism to ensure secure user access and prevent unauthorized registrations. MySQL is used as the database layer to store user details, event information, registrations, attendance records, seat bookings, and certificates. This framework ensures scalability, security, and efficient performance for the Smart Campus Event Management System.

The Smart Campus Event Management System follows a layered architecture consisting of the Presentation Layer, Application Layer, and Data Layer.

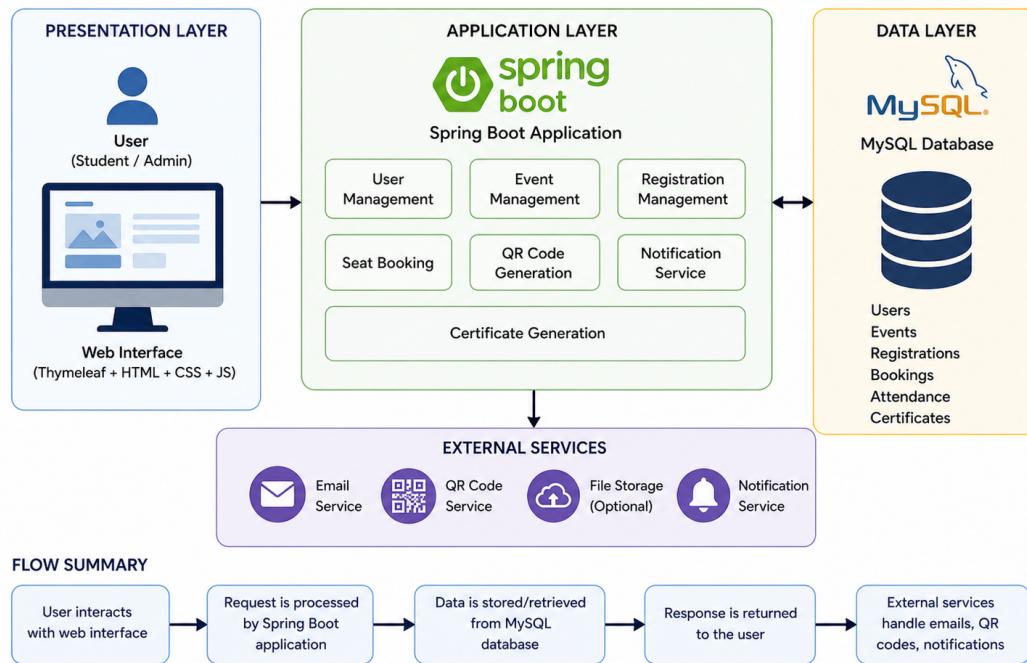


Figure 1.1: System Architecture

The Presentation Layer provides the user interface where students and administrators interact with the system using Thymeleaf, HTML, CSS, and JavaScript. The Application Layer is developed using Spring Boot and manages core functionalities such as user management, event handling, registration, seat booking, QR code generation, certificate generation, OTP-based verification, and notification services. The Data Layer uses MySQL to store information related to users, events, registrations, attendance, and certificates. External services such as email notifications and QR code generation are also integrated. This architecture ensures efficient processing, security, and smooth communication between all system components.

Chapter 2

SURVEY OF SIMILAR APPLICATIONS IN THE MARKET

Several existing platforms provide event management services, but they are often designed for general-purpose use and may not fully meet the specific requirements of educational institutions:

- **Eventbrite** is a widely used platform for public event registration and ticket booking. It allows organizers to create events, sell tickets, and manage attendees. However, it is primarily designed for commercial events and lacks features specific to campus environments such as attendance tracking and academic reporting.
- **BookMyShow** provides seat booking functionality mainly for movies, concerts, and entertainment programs. While it offers efficient seat allocation and booking features, it does not support academic event management functionalities such as student registration tracking, certificate generation, or feedback collection.
- **Google Forms** is commonly used in colleges for simple event registrations. It is easy to use and accessible, but it lacks automation features such as real-time seat booking, attendance verification,

and certificate generation. Data management also becomes difficult as the number of responses increases.

- **Manual Registration Systems** involve paper-based registration methods or basic spreadsheet management. These methods are time-consuming, error-prone, and inefficient, leading to issues such as duplicate entries, poor communication, and difficulty in maintaining accurate records.

2.1 Limitations of Existing Systems:

- Lack of college-specific customization for academic needs
- No integrated attendance tracking mechanism
- Absence of automatic certificate generation
- Inefficient communication between organizers and students
- Limited automation and higher chances of human error

2.2 Our Proposed System Advantages:

- Real-time seat booking system for better resource management
- QR code-based event entry and attendance tracking
- Automatic certificate generation for participants

- Email notifications for updates and confirmations
- Integrated student feedback system
- Google Maps integration for easy venue navigation
- Implements OTP-based verification during ticket booking.

Table 2.1: Feature Comparison of Existing Applications

Feature	Google Forms	Eventbrite	Manual System	Our System
Online Registration	Yes	Yes	No	Yes
Seat Booking	No	Yes	No	Yes
Attendance Tracking	No	Limited	No	Yes
Certificate Generation	No	No	No	Yes
Notifications	No	Yes	No	Yes
Feedback System	No	Limited	No	Yes

Table 2.2: System Comparison Based on Functionality

System	Description
Google Forms	Basic registration without automation
Eventbrite	Public event platform with ticket booking
Manual System	Paper-based and time-consuming process
Our System	Complete automated campus event management

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

- **HTML** Used to create the basic structure of web pages such as login, registration, dashboard, and event pages.
- **CSS** Used for styling the web pages and improving the visual appearance of the application.
- **JavaScript** Used to add interactivity such as form validation, dynamic updates, and seat selection functionality.
- **Bootstrap** Used to design responsive and mobile-friendly user interfaces with pre-built components.
- **Thymeleaf Templates** Used as a server-side template engine to integrate frontend pages with backend data in the Spring Boot application.

3.1.2 Structure of the Project

- **login.html** Allows users to securely log in to the system using their credentials.
- **register.html** Enables new students/users to create an account in the system.
- **dashboard.html** Displays an overview of events, notifications, and user activities.
- **events.html** Shows the list of available events and allows students to register.
- **seat-booking.html** Provides real-time seat selection functionality for event booking.
- **feedback.html** Allows students to submit feedback after attending events.

3.1.3 Execution Procedure

The frontend of the Smart Campus Event Management System is developed using HTML, CSS, JavaScript, and Thymeleaf templates. It is integrated with the Spring Boot backend and rendered through the server. The execution procedure is as follows:

1. **Environment Setup:** Install a web browser such as Google Chrome and a code editor like Visual Studio Code for development.
2. **Project Structure Setup:** Frontend files are organized within the `src/main/resources` directory. HTML templates are placed in the `templates/` folder, while CSS, JavaScript, and images are stored in the `static/` folder.
3. **Template Development:** User interface pages such as login, registration, event listing, and dashboard are developed using HTML and Thymeleaf for dynamic content rendering.
4. **Styling and Layout:** CSS is used to design responsive and user-friendly interfaces. Consistent layout and styling are maintained across all pages.
5. **Client-side Scripting:** JavaScript is used to enhance interactivity, perform form validation, and handle dynamic behaviors on the client side.
6. **Integration with Backend:** Thymeleaf templates are integrated with Spring Boot controllers to dynamically display data such as events, user details, and registration status.
7. **Application Execution:** When the backend server is started,

the frontend pages are automatically served and can be accessed through `http://localhost:8080`.

8. **Testing and Validation:** All frontend pages are tested for correct navigation, proper data rendering, and responsiveness across different screen sizes.

3.2 Backend Implementation

3.2.1 Environment Setup

- **Java 17** Used as the primary programming language for developing the backend application.
- **Spring Boot** Simplifies backend development by providing built-in configurations and dependency management.
- **Spring Security** Handles authentication and authorization for secure user access.
- **Maven** Used for project build management and dependency handling.
- **REST APIs** Enables communication between frontend and backend components through HTTP requests.

3.2.2 Structure of the Project

The project is divided into multiple layers, each responsible for a specific functionality:

- **Controller Layer:** Handles incoming HTTP requests and maps them to appropriate backend services. It acts as the entry point of the application.
- **Service Layer:** Contains the core business logic of the system. It processes user requests, applies rules, and interacts with the repository layer.
- **Repository Layer:** Responsible for database operations using Spring Data JPA. It abstracts CRUD operations and interacts with the database efficiently.
- **Model Layer:** Defines entity classes that represent database tables such as users, events, and registrations.
- **Resources Folder:** Contains configuration files (`application.properties`), static resources (CSS, JavaScript), and template files (HTML using Thymeleaf).
- **Main Class:** The `SmartCampusApplication.java` file serves as the entry point of the Spring Boot application.

- **Test Folder:** Includes unit and integration test cases to ensure system reliability.

3.2.3 Execution Procedure

The execution of the backend for the Smart Campus Event Management System involves a series of steps to ensure proper setup, configuration, and successful deployment of the application. The backend is developed using the Spring Boot framework and follows a structured execution flow as described below:

1. **Environment Setup:** Install Java Development Kit (JDK 8 or above) and Apache Maven. Configure environment variables such as `JAVA_HOME` and `MAVEN_HOME`.
2. **Project Import:** Import the project into an Integrated Development Environment (IDE) such as Eclipse or IntelliJ IDEA as a Maven project.
3. **Dependency Installation:** Maven automatically downloads all required dependencies specified in the `pom.xml` file.
4. **Database Configuration:** Configure database connection details (URL, username, password) in the `application.properties` file.

5. **Build the Project:** Run the following command to clean and build the project:

```
mvn clean install
```

6. **Run the Application:** Execute the Spring Boot application using:

```
mvn spring-boot:run
```

or run the main class `SmartCampusApplication.java` from the IDE.

7. **Server Initialization:** The embedded server (Tomcat) starts automatically, and the application is deployed on `http://localhost:8080`.

8. **API Testing:** Test backend APIs using tools such as Postman or a web browser to verify proper functionality.

9. **Logging and Monitoring:** Check application logs for debugging and performance monitoring.



Figure 3.1: Execution Procedure

The execution procedure of the Smart Campus Event Management System begins with the user accessing the system through login authentication. After successful login, users can browse available events and view details such as date and venue. Students then register for events and proceed with real-time seat booking by selecting available seats. If applicable, payment is completed to confirm the booking. Once registration is successful, the system generates a confirmation along with a QR code, which serves as a digital entry pass. During the event, attendance is marked by scanning the QR code, and after verification, certificates are automatically generated for participants. This process ensures a smooth, secure, and efficient event management workflow.

3.3 Database Implementation

3.3.1 Database Configuration

The database for the Smart Campus Event Management System is configured using Spring Boot and managed through Spring Data JPA. The configuration is defined in the `application.properties` file, which establishes the connection between the application and the relational database.

The following parameters are used for database configuration:

- **Database URL:** Specifies the location of the database.

```
spring.datasource.url  
=jdbc:mysql://localhost:3306/smart_campus
```

- **Database Username:** Defines the username for database access.

```
spring.datasource.username=root
```

- **Database Password:** Defines the password for database authentication.

```
spring.datasource.password=2648
```

- **JPA Configuration:** Specifies Hibernate behavior for database operations.

```
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect
=org.hibernate.dialect.MySQLDialect
```

- **Driver Configuration:** Specifies the JDBC driver for MySQL.

```
spring.datasource.driver-class-name
=com.mysql.cj.jdbc.Driver
```

The configuration enables automatic table creation and updates using Hibernate. It also allows the application to interact with the database efficiently through repository interfaces.

The database schema includes tables such as *User*, *Event*, and *Registration*, which are mapped using JPA entity classes in the model layer.

3.3.2 Schema Structure

- **Student** Stores student details such as name, email, login credentials, and profile information.
- **Admin** Stores administrator details used to manage events and users.
- **Event** Contains information about events such as title, date, venue, and description.

- **Registration** Maintains records of students registered for specific events.
- **Seat** Stores seat allocation details for event booking.
- **Feedback** Collects feedback submitted by students after attending events.
- **Attendance** Tracks student participation and attendance status.
- **Certificate** Stores certificate details generated for event participants.

3.3.3 Tables Created

- **student** Stores student account and personal details.
- **admin** Stores administrator login and management details.
- **event** Contains event-related information.
- **registration** Stores event registration records.
- **seat** Maintains seat booking information.
- **feedback** Stores feedback submitted by students.
- **attendance** Tracks attendance records for events.
- **certificate** Stores generated certificate details.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The system follows a three-tier architecture:

- **Presentation Layer** This layer provides the user interface where students and administrators interact with the system through web pages.
- **Business Logic Layer** This layer processes user requests, applies business rules, and manages application functionalities.
- **Database Layer** This layer stores and retrieves data related to students, events, registrations, attendance, and certificates.

4.2 DataFlow Diagram

The data flow of the Smart Campus Event Management System follows the process where students/admins interact with the frontend, requests are processed through backend APIs, data is stored/retrieved from the database, and responses are returned to users.

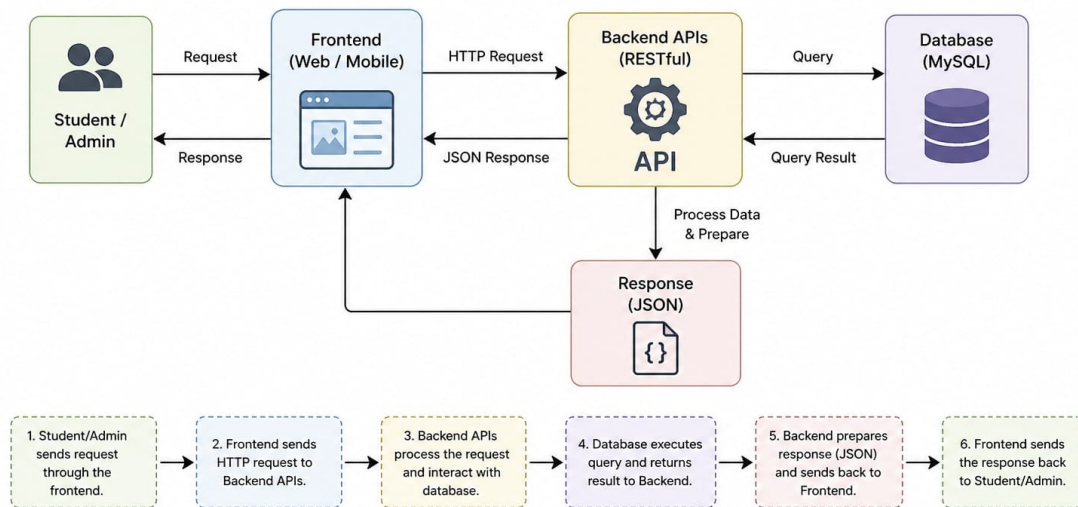


Figure 4.1: Data Flow Diagram

The data flow of the Smart Campus Event Management System begins when a student or admin sends a request through the frontend interface. The frontend forwards it as an HTTP request to the backend APIs. The backend, developed using Spring Boot, handles the request, applies business logic, and interacts with the MySQL database to retrieve or store the required data. The database executes the query and returns the result to the backend. The backend then prepares a structured JSON response and sends it back to the frontend, which displays the information to the user. This flow ensures efficient communication, real-time updates, and secure data handling within the system.

4.3 Module 1: Authentication Module

- **Student Registration** Allows new students to create an account in the system.
- **Login** Enables registered users to securely access their accounts.
- **Role Management** Differentiates access privileges between students and administrators.
- **OTP Verification** Ensures secure login and ticket booking by sending a one-time password to the user's registered email.
- **Security** Protects user data through authentication and authorization mechanisms using Spring Security.

4.4 Module 2: Event Management Module

- **Create Events** Allows administrators to add new events with details such as title, date, venue, and description.
- **Update Events** Enables administrators to modify existing event information when required.
- **Delete Events** Allows administrators to remove cancelled or outdated events from the system.

- **Registration Management** Helps administrators monitor and manage student registrations for different events.

4.5 Module 3: Smart Features Module

- **Real-time Seat Booking** Allows students to select available seats dynamically during event registration.
- **QR Code Generation** Generates QR codes for event entry verification and attendance tracking.
- **Certificate Generation** Automatically provides participation certificates to students after event completion.
- **Email Notifications** Sends confirmation emails and event updates to registered students.
- **OTP-based Ticket Verification** Enhances security by validating user identity during event registration and seat booking.
- **Chatbot Recommendations** Suggests relevant events and assists users by answering basic queries.
- **Google Maps Integration** Displays event venue locations using Google Maps to help students easily navigate to event locations.

4.5.1 Advantages of this Project

- **Automation** Reduces manual work involved in event registration, seat allocation, attendance tracking, and certificate generation, thereby saving time and effort.
- **Improved User Experience** Students can easily browse events, register online, and book seats in real-time through a simple and user-friendly interface.
- **Enhanced Security** Role-based authentication along with OTP verification ensures secure access and prevents unauthorized registrations.
- **Real-time Seat Booking** Allows users to select seats dynamically based on availability, avoiding overbooking and confusion.
- **Efficient Communication** Email notifications and updates help keep students informed about event details, confirmations, and changes.
- **Paperless System** Reduces the use of paper by enabling digital registration, attendance tracking, and certificate generation.
- **Centralized Management** All event-related data is stored and managed in a single system, making it easier for administrators to monitor and control activities.

4.5.2 Disadvantages

- **Requires Internet Connection** Users need a stable internet connection to access the system, which may be a limitation in areas with poor connectivity.
- **Initial Setup Cost** Development, deployment, and hosting of the system may require initial investment.
- **Requires Technical Maintenance** Regular updates, bug fixes, and system maintenance are necessary to ensure smooth performance and security.
- **Dependency on Technology** System performance depends on server availability and technical infrastructure, which may cause disruptions if failures occur.
- **Learning Curve** New users may require some time to understand and adapt to the system features and functionalities.

Chapter 5

RESULTS AND DISCUSSIONS

The project was successfully implemented and tested. Students were able to register for events, book seats in real-time, receive QR passes, and download certificates after attendance verification. The system also successfully implemented OTP-based verification, where one-time passwords were reliably generated, delivered to users, and validated during registration and ticket booking. Admin users were able to create events, monitor registrations, verify attendance, and collect student feedback. The system performed efficiently with minimal response time and improved overall event management operations.

Table 5.1: Comparison of Existing Applications

Existing System	Limitation
Google Forms	No seat booking
Eventbrite	Not college-specific
Manual Registration	Time-consuming
Our System	Complete automation

5.1 Login Page

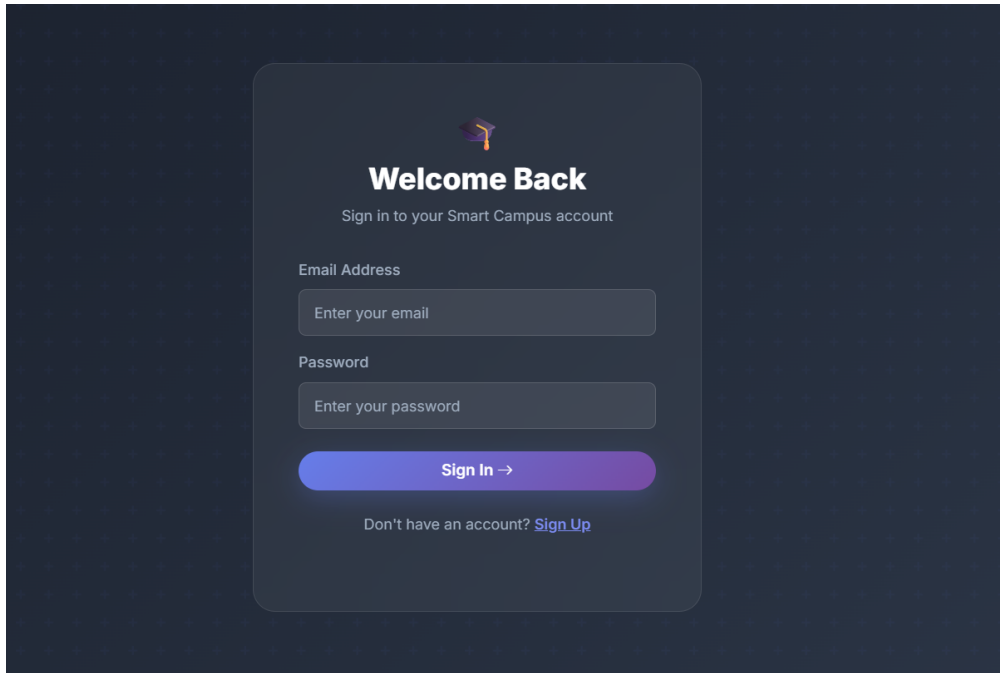


Figure 5.1: Login Page

The Login Page is the entry point of the Smart Campus Event Management System, where users authenticate themselves to access the platform. It provides a simple and user-friendly interface that allows students and administrators to enter their registered email address and password. The system validates the credentials using secure authentication mechanisms implemented through Spring Security. Upon successful login, users are redirected to their respective dashboards based on their roles. The login page also supports account registration for new users and ensures secure access to the system, maintaining data privacy and protection.

5.2 Dashboard

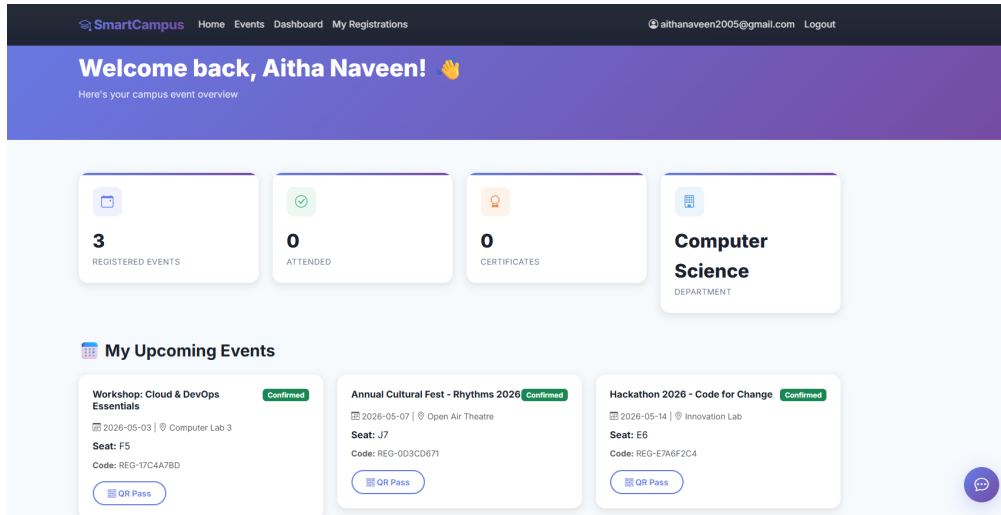


Figure 5.2: Dashboard

The Dashboard provides an overview of user activities and event-related information in the Smart Campus Event Management System. After successful login, users are redirected to the dashboard, where they can view key details such as the number of registered events, attended events, and certificates earned. It also displays user-specific information like department details and personalized greetings. The dashboard includes a section for upcoming events, showing event names, dates, venues, seat numbers, and registration status. Users can access their QR passes directly for event entry and attendance verification. This centralized interface enables users to easily track their participation and manage their event activities efficiently.

5.3 Event Listing

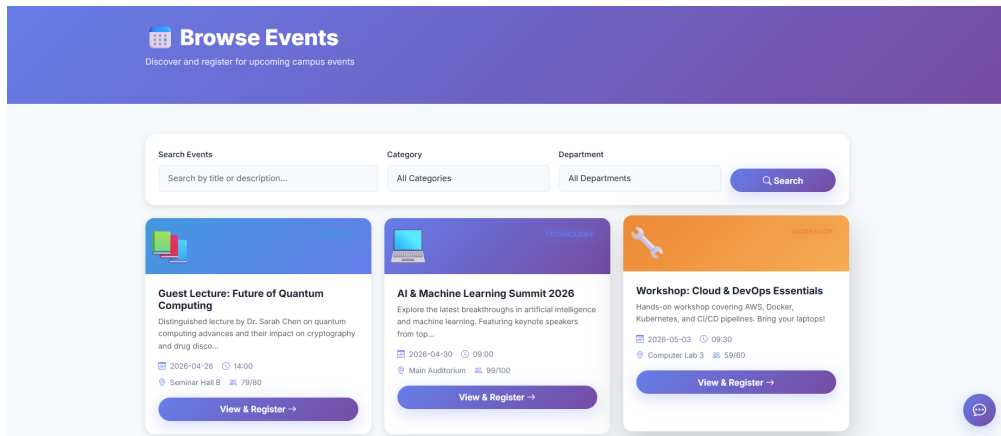


Figure 5.3: Event Listing

The Event Listing page allows users to explore and register for various campus events in the Smart Campus Event Management System. It displays a list of upcoming events with details such as event title, description, date, time, venue, and available seats. Users can filter events based on categories and departments or search using keywords to quickly find relevant events. Each event card provides a "View & Register" option, enabling users to access detailed information and proceed with registration. This feature ensures easy navigation, efficient event discovery, and a smooth registration experience for users.

5.4 Seat Selection and Booking with OTP Verification

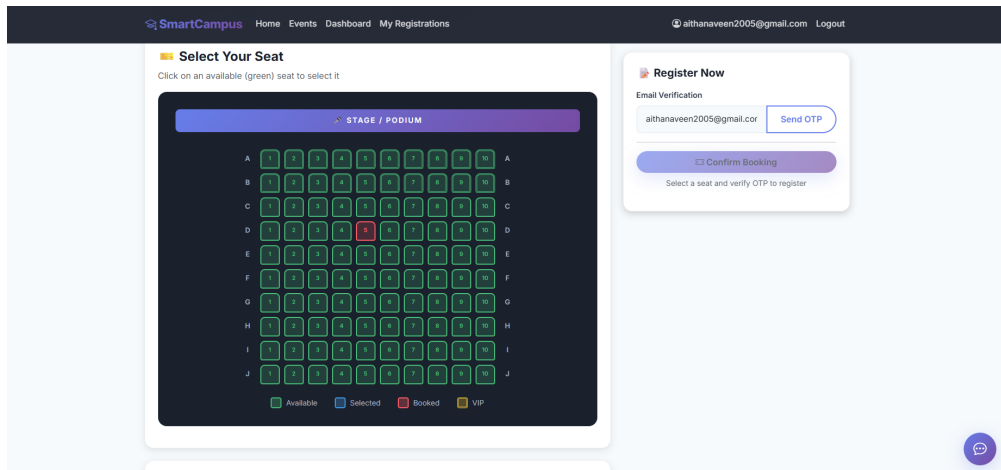


Figure 5.4: Seat Selection and Booking with OTP Verification

The Seat Selection and Booking module allows users to choose their preferred seats in real-time during event registration. The system displays a seating layout where available seats are highlighted, and users can select seats based on availability. Once a seat is selected, the user proceeds to the booking section, where OTP-based verification is implemented for security. A one-time password (OTP) is sent to the user's registered email or mobile number, and the booking is confirmed only after successful OTP validation. This process prevents unauthorized bookings and ensures that only genuine users can reserve seats. The module provides a seamless and secure booking experience while avoiding seat conflicts and overbooking.

5.5 Venue Location

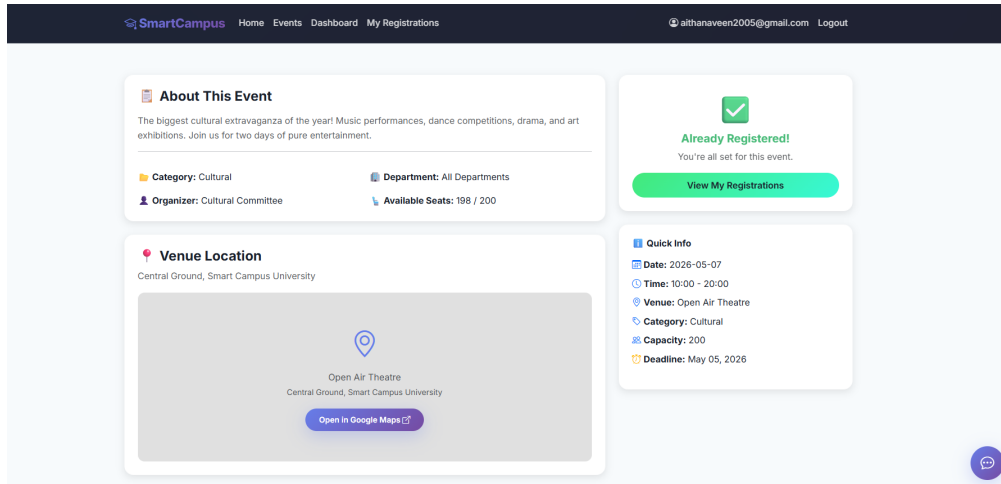


Figure 5.5: Venue Location

The Venue Location feature provides users with accurate details about the event location within the Smart Campus Event Management System. It displays essential information such as venue name, address, date, time, and event category. The system integrates Google Maps to help users easily locate the event venue and navigate without confusion. Users can directly access the location through a map interface or an external link to Google Maps for real-time directions. This feature enhances user convenience, reduces the chances of missing events due to location issues, and improves overall event accessibility.

5.6 Admin Panel

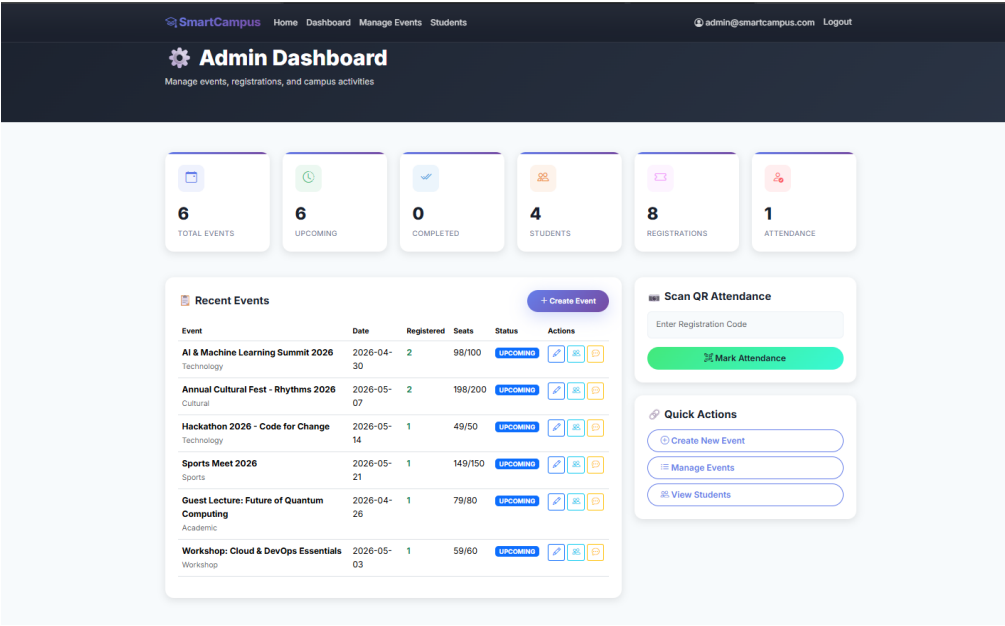


Figure 5.6: Admin Panel

The Admin Panel provides a centralized interface for administrators to manage all event-related activities in the Smart Campus Event Management System. It displays key statistics such as total events, upcoming events, completed events, registered students, and attendance records. Administrators can create, update, and delete events, as well as monitor registrations and seat availability. The panel also includes features for scanning QR codes to mark attendance efficiently and accurately. Additionally, quick action options allow admins to manage events and view student details with ease. This module ensures efficient control, better coordination, and streamlined management of campus events.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Smart Campus Event Management System has been successfully designed and implemented to automate and streamline event management activities within educational institutions. The system effectively replaces traditional manual methods such as paper-based registrations and spreadsheet tracking with a centralized digital platform.

Through this application, students can easily browse available events, register online, book seats in real-time, and access important information such as event schedules and venue locations. Administrators are provided with powerful tools to create and manage events, monitor registrations, track attendance using QR codes, and generate participation certificates efficiently.

The integration of modern technologies such as Spring Boot, Thymeleaf, and MySQL ensures that the system is secure, scalable, and reliable. The use of role-based authentication enhances system security, while automation reduces human errors and administrative

workload.

Overall, the project improves operational efficiency, enhances user experience, and promotes greater student engagement in campus activities. It serves as a practical solution for modern educational institutions aiming to digitize their event management processes.

6.2 Future Enhancements

Although the current system provides a comprehensive solution, several enhancements can be implemented to further improve its functionality and usability:

- **Mobile Application Development** A dedicated mobile application can be developed to provide users with convenient access to event services anytime and anywhere. This will improve accessibility and user engagement through push notifications and mobile-friendly interfaces.
- **AI Event Recommendation System** Artificial Intelligence techniques can be integrated to analyze user preferences and past participation data to recommend relevant events. This personalized approach can increase student involvement and improve event participation rates.
- **Payment Integration** Online payment gateways can be integrated

to support paid events. This will allow users to securely pay registration fees and enable organizers to manage transactions efficiently.

- **Face Recognition Attendance** Facial recognition technology can be implemented to automate attendance tracking. This reduces manual verification efforts and ensures accurate and secure attendance recording.
- **Cloud Deployment** The system can be deployed on cloud platforms such as AWS or Google Cloud. Cloud deployment improves scalability, reliability, and accessibility, allowing the system to handle a larger number of users and events.
- **Advanced Analytics Dashboard** An advanced analytics module can be added to provide detailed insights into event participation, user engagement, and system performance. This helps administrators make data-driven decisions and improve future event planning.

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

Table 7.1: Mapping of Project to Complex Engineering Problem (CEP)

Level	Attribute	Characteristics	Wks	Justification
WP1	Depth of knowledge	Requires in-depth engineering knowledge	WK3, WK4, WK5, WK6	Uses Spring Boot, MVC architecture, database design, REST APIs, and frontend technologies.
WP2	Conflicting requirements	Technical and non-technical conflicts	WK4, WK5, WK6	Balances usability and security with authentication and validation.
WP3	Depth of analysis	Analytical and design thinking	WK3, WK4, WK5	Includes system design, database schema, validation, and event filtering.
WP4	Familiarity	Partially new problems	WK4, WK8	Real-time updates and admin analytics add complexity.
WP5	Applicable codes	Limited standard solutions	WK6	Requires custom implementation for event handling and tracking.
WP6	Stakeholder needs	Multiple stakeholders involved	WK5, WK6, WK7	Supports students, faculty, and administrators.
WP7	Interdependence	System-level integration	WK3, WK5, WK6	Integrates frontend, backend, database, and security modules.

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

Table 7.2: Mapping of Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification
EA1	Range of Resources	Use of diverse technologies	Uses Spring Boot, Spring MVC, Spring Data JPA, Thymeleaf, HTML/CSS, and relational databases for full-stack development.
EA2	Level of Interactions	Complex interactions between modules	Manages interactions between event management, user registration, validation, authentication, and reporting modules.
EA3	Innovation	Application of modern technologies	Implements a web-based centralized platform with filtering, real-time updates, and secure admin operations.
EA4	Consequences to Society and Environment	Impact on users and society	Enhances student engagement and improves institutional efficiency while reducing manual errors and delays.
EA5	Familiarity	Beyond standard practices	Combines web development, security, and data analytics into a unified system.

7.3 SDG Mapping

Table 7.3: SDG Mapping for the Project

SDG No.	SDG Title	Relevance
SDG 4	Quality Education	Enhances student participation in academic events.
SDG 9	Industry, Innovation and Infrastructure	Promotes digital transformation in institutions.
SDG 11	Sustainable Cities and Communities	Supports smart campus initiatives.
SDG 16	Peace, Justice and Strong Institutions	Ensures secure and transparent system.
SDG 12	Responsible Consumption and Production	Reduces paper usage through digital system.

Chapter 8

REFERENCES

- Spring Boot Official Documentation <https://spring.io/projects/spring-boot>
- MySQL Official Documentation <https://dev.mysql.com/doc/>
- Thymeleaf Official Documentation <https://www.thymeleaf.org/documentation.html>
- Spring Security Official Documentation <https://spring.io/projects/spring-security>
- Bootstrap Documentation <https://getbootstrap.com/docs/>
- HTML and CSS Documentation (MDN Web Docs) <https://developer.mozilla.org/>
- REST API Design Guide <https://restfulapi.net/>
- GitHub Project Repository <https://github.com/aithanaveen/Smart-Campus-Event-Management>